

Secure coding

Learning objectives

- **Define secure coding**
- **Explain why secure coding is critical to cybersecurity**
- **Identify common causes of insecure code**
- **Describe key secure coding practices**
- **Identify who's responsible for writing secure code**

Secure coding

- **Writing code that avoids security flaws**
- **Prevents vulnerabilities from the start**
- **Resilient to misuse and attacks**
- **Essential part of professional software development**

Why does secure coding matter?

Insecure code can lead to:

- **Data breaches**
- **Account takeovers**
- **Malware injection**
- **System downtime**
- **Legal and compliance failures**

What causes insecure code?

- **Lack of security awareness**
- **Time pressure or shortcuts**
- **Copy-pasted code from the internet**
- **Misuse of APIs or libraries**
- **Incomplete input validation**
- **No secure coding training**

The real-world impact of insecure code

Examples from industry:

- **Social media breach: hardcoded administrator password**
- **Healthcare app: exposed patient data**
- **Online shop: customer card data theft**
- **Government site: downtime from unhandled errors**

Core principles of secure coding

- **Security by Design**
- **Least Privilege**
- **Input Validation**
- **Output Encoding**
- **Fail Securely**
- **Avoid Hiding Security Details**
- **Keep It Simple**

Best practices in secure coding

Developers should:

- **Validate and sanitise input**
- **Use prepared statements for queries**
- **Implement proper authentication**
- **Avoid hardcoded credentials**
- **Choose secure libraries**
- **Handle errors safely**
- **Review and test code for security**

Who is responsible for secure code?

It's a shared responsibility

- **Developers: write clean, secure code**
- **Team leads: promote best practices**
- **Testers: identify issues**
- **Architects: design safe systems**
- **DevOps: secure the environment**
- **Security teams: provide guidance**

Secure coding in the software lifecycle

- **Requirements: define security goals**
- **Design: apply secure architecture**
- **Development: follow secure coding practices**
- **Testing: include security tests**
- **Deployment: lock down configurations**
- **Maintenance: monitor and patch**

Summary

- **Secure coding prevents major risks**
- **Simple mistakes can lead to big consequences**
- **Everyone on the team has a part to play**

Next

- **Applying secure coding throughout the development process.**
- **Focusing on secure file handling and configuration practices.**